Contents

I Engineering and Execution **2**

1 The host, the data path, and the training stack **3**

1.1 The host and its constraints 3

1.2 The data path 4

1.3 The GPU-resident input path 5

1.4 Determinism 5

1.5 What every run leaves behind 5

2 Scheduling, execution, and archiving **7**

2.1 Scheduling, failure handling, and collection 7

2.2 Execution 7

2.3 Archiving and vacating 10

3 Errors, setbacks, and fixes **11**

PART I

Engineering and Execution

1

The host, the data path, and the training stack

1.1 The host and its constraints

All Study 2 and Amendment 8 computation ran on one shared server (`node1`) with eight NVIDIA RTX A6000 GPUs (49 GB each). It has no job scheduler; work runs directly on the login node. The operating rules for the project were: never more than four GPUs at once, check for vacant GPUs first, keep data that feeds GPUs off the shared network filesystem, use the fast local disk and vacate it afterwards.

Table 1.1: Host constraints discovered or imposed, and the response to each.

Constraint	Consequence and response
Four GPUs at most	Study 2 used GPUs 0, 3, 4, 5 (GPUs 1 and 2 were in use by another user); Amendment 8 used 4–7 after another user started jobs on 0–3 twenty-five minutes before launch. The scheduler refuses more than four.
Four CPU cores per user	Found during Study 2 preparation: the per-user cgroup quota is 400% (four cores) on a 128-core machine. The power simulation ran far slower than its core count suggested, a data-loader benchmark was invalid, and per-image augmentation in loader workers could not feed even one GPU. This is very likely the "host bottleneck" recorded in Study 1 (Chapter 15). Response: decode crops once into caches, upload them to the GPU, augment there (Section 1.3), keep CPU threads low, and run heavy CPU analyses on a workstation.
`/scratch` not writable	All node-local work used `/tmp/rchethan1_cytofair` (the 192 GB root partition shared with the operating system) and `/dev/shm`; a disk guard keeps 40 GB free.
Slow network home	The NFS home can stall under load; a recursive search over it had to be stopped early in Study 2 preparation. Rule adopted: only static artifacts go there, and copies are single bandwidth-limited streams.

Constraint	Consequence and response
Shared GPUs	Other users' jobs can appear at any time. The scheduler checks each GPU's memory before launching (at most 30,000 MiB in use), and an infrastructure failure caused by another process does not count as a retry (Amendment 7).
No scheduler	Background controllers launched with `setsid nohup` and detached from the SSH session; a status file records every running task, completion, and failure.

1.2 The data path

1.2.1 Provenance to splits

Every split is derived from the provenance maps of Study 1 (`provenance/*.parquet`), whose recovery and verification are described in Part II. `cytofair study2-design` writes every Study 2 split, factorial design, transplant plan, and the probe split with SHA-256 hashes into `splits/study2/study2_split_manifest.csv`; `cytofair followup-design` does the same for Amendment 8 (`splits/study2/followup/`). Trainers hash the split file before use and refuse a mismatch.

1.2.2 Packed crop caches

Decoding thousands of small BMP and PNG files on four cores is slow, and reading them from the network home is unreliable. Each image set is therefore packed once into a cache on the node-local disk: one `uint8` array of all pixels (`{name}_pixels.npy`, memory-mapped), an index (`{name}_index.parquet`) with each crop's offset, height, width, and the SHA-256 of its pixels, and a manifest with the SHA-256 of both files. A cache is verified on load; a hash mismatch stops the run.

Table 1.2: Caches used by Study 2 and Amendment 8 (hashes in Appendix P).

Cache	Images	Content
`sipakmed`	4,049	every SIPaKMeD crop
`cric`	11,534	100×100 windows around every CRIC nucleus
`transplant_seed*` (10)	4,096–4,400 each	2B composites of each seed's common test set
`sipakmed_masked`	4,049	background set to the ImageNet mean (F3)
`sipakmed_reinhard50/100`	4,049 each	Reinhard normalisation at strength 0.5 and 1 (F6)
`ablation_seed*` (10)	six per test crop	the F2 carrier ablations

1.3 The GPU-resident input path

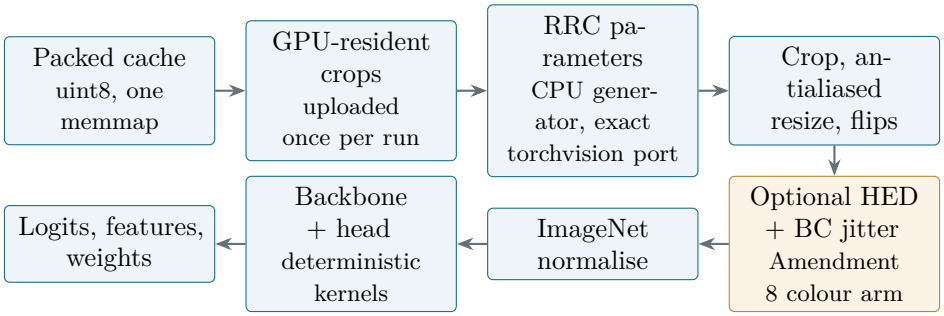


Figure 1.1: The GPU-resident training input path. The host allowed four CPU cores per user, which made per-image PIL augmentation in data-loader workers the bottleneck; moving the registered augmentation onto the GPU removed it while sampling exactly the same crop geometry as torchvision for the same random stream.

The registered augmentation is `RandomResizedCrop(224, scale 0.8-1.0, ratio 3/4-4/3)`, horizontal and vertical flips, and ImageNet normalisation. `src/cytofair/gpu_augment.py` keeps every crop of a split on the GPU as a `uint8` tensor and draws crop parameters with an exact port of torchvision's `RandomResizedCrop.get_params` from an explicit CPU generator, including the float32 exponential torchvision uses for the aspect ratio. A unit test compares the sampled geometry with torchvision's for four image shapes and 25 seeds each and requires identity. Resizing uses antialiased bilinear interpolation, PyTorch's implementation of PIL's filter, which can differ from PIL by uint8 rounding. The Amendment 8 colour arm adds HED optical-density jitter and brightness/contrast jitter, drawn from the same generator after the geometry, so runs without colour augmentation reproduce the registered random stream exactly.

1.4 Determinism

Every run sets cuDNN deterministic mode, disables cuDNN benchmarking, calls `torch.use_deterministic_algorithms(True,warn_only=True)`, and sets `CUBLAS_WORKSPACE_CONFIG=:4096:8`. During smoke tests ViT-B/16 raised a warning that the memory-efficient attention backward pass is non-deterministic; the flash, memory-efficient, and cuDNN attention kernels were disabled in favour of the math kernel, and two identical ViT runs then produced bitwise-identical logits. The follow-up evaluation code avoids `index_add_` (non-deterministic on CUDA) by computing group sums as one-hot matrix products.

Final weights are stored in float16 to halve their size. Re-evaluating the saved Phase 1 checkpoints reproduced the recorded test accuracy exactly for 418 of 480 models; 56 differed by one test crop and six by two or three crops out of 810, all from float16 rounding. Every follow-up comparison is made within one re-evaluation, so this does not bias any registered contrast.

1.5 What every run leaves behind

`src/cytofair/train_study2.py` writes, for every run:

- a JSON record with the dataset, protocol, architecture, seed, accuracy, macro-F1, balanced accuracy, NLL, 15-bin ECE, the confusion matrix, train and test crop and parent counts, the fraction of test crops with a training parent, the recipe and augmentation, adversary settings,

the split file and semantic hashes, cache hashes, the git commit, GPU, start and end times, runtime metadata, a per-epoch history, and the path and SHA-256 of every artifact;

- per-crop test logits (Parquet), float16 final weights, penultimate features of the 1,696 probe crops (NPZ), and logits for any registered extra evaluation set (the 2B composites).

The follow-up evaluation tasks write comparable records (`src/cytofair/followup_eval.py`), verifying each checkpoint's hash before loading it. The exported tables in `results/study2/run_records` keep every field the record plots, including all 45,600 epoch histories.

2

Scheduling, execution, and archiving

2.1 Scheduling, failure handling, and collection

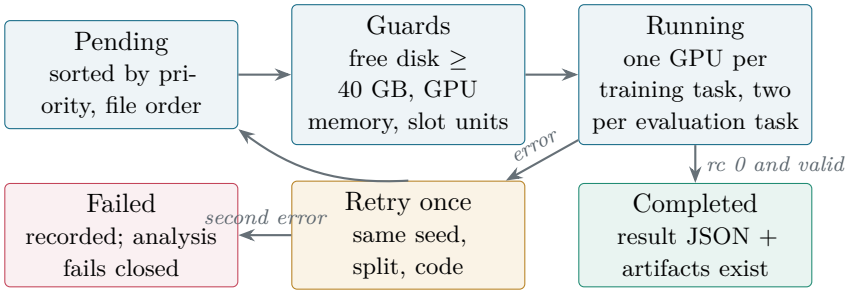


Figure 2.1: The resumable scheduler (`scripts/study2/launch_matrix.py`). A task whose valid result already exists is skipped on restart, so the controller can be relaunched after any interruption without duplicating a run.

A task is a JSON line naming its command, the result file that proves completion, a priority, the GPU share it needs, and an `expect` block of fields its result must carry (dataset, protocol, architecture, seed, split hash, and for the follow-up arms the cache, colour augmentation, and adversary). `scripts/study2/build_tasks.py` wrote the 1,341 Study 2 tasks and `scripts/study2/build_followup_tasks.py` the 1,580 follow-up tasks; both sets are committed and hashed. Training tasks took a whole GPU; evaluation-only tasks took half, so two ran per GPU.

The collectors (`analysis/study2/collect_runs.py`, `analysis/study2/followup/prepare_followup.py`) require exactly one valid result per registered task whose recorded fields match its `expect` block; anything missing, duplicated, or mismatched is listed and the analysis stops. `-allow-incomplete` produces progress reports only. Under Amendment 7, adversarial runs whose every attempt ended in a non-finite loss are recognised from their logs and recorded as diverged rather than missing. Neither matrix had a failure, retry, or divergence.

2.2 Execution

2.2.1 Smoke tests

Before each registration, short runs exercised every code path on splits that cannot reveal a registered contrast: P1 splits only for Study 2; leak cells, single P1 runs, and single evaluation tasks

for Amendment 8 (Table M.6). Their metrics were checked only for finiteness and integrity (for example, that a re-evaluated checkpoint reproduces its recorded accuracy).

2.2.2 The two matrices

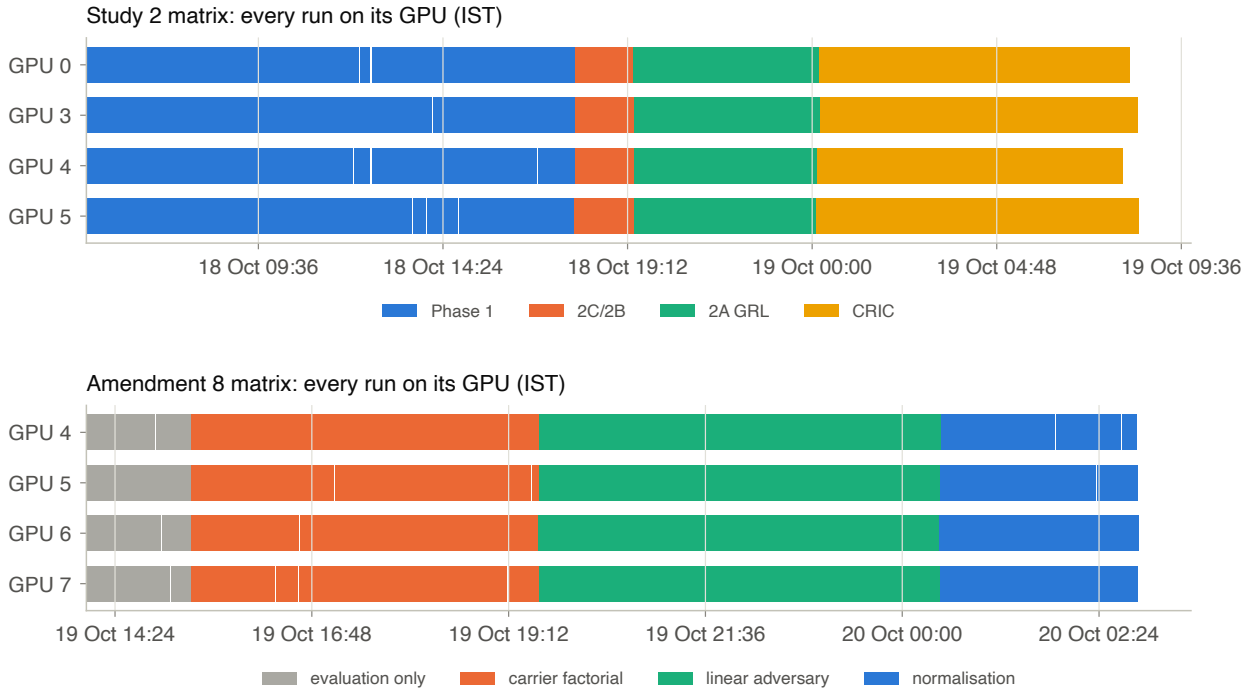


Figure 2.2: Every run of both matrices on its GPU, from the start and end times in the run records (evaluation tasks from the controller log). The arms ran in priority order; white gaps are launch overhead.

Table 2.1: Compute ledger of every Study 2 and Amendment 8 arm (node1, NVIDIA RTX A6000; times in IST). GPU-hours are summed per-run elapsed time; runs shared four GPUs.

Arm	Runs	Mean min	GPU-h	First start	Last finish
Study 2 Phase 1 (H1)	720	3.9	47.1	18 Oct 05:07	18 Oct 17:50
Study 2 2C/2B (H3, H4)	160	2.0	5.4	18 Oct 17:48	18 Oct 19:21
Study 2 2A (H5)	280	3.9	18.2	18 Oct 19:20	19 Oct 00:12
Study 2 CRIC (H2)	180	10.7	32.0	19 Oct 00:06	19 Oct 08:28
A8 carrier factorial (F3, F4)	480	1.9	15.2	19 Oct 15:19	19 Oct 19:34
A8 linear adversary (F7)	300	3.7	18.5	19 Oct 19:33	20 Oct 00:27
A8 stain normalisation (F6)	160	3.4	9.1	20 Oct 00:27	20 Oct 02:52
A8 ablations + Grad-CAM (F2)	160	0.2	0.6	19 Oct 14:03	19 Oct 14:14
A8 LEACE erasure (F5)	480	0.8	6.5	19 Oct 14:13	19 Oct 15:19

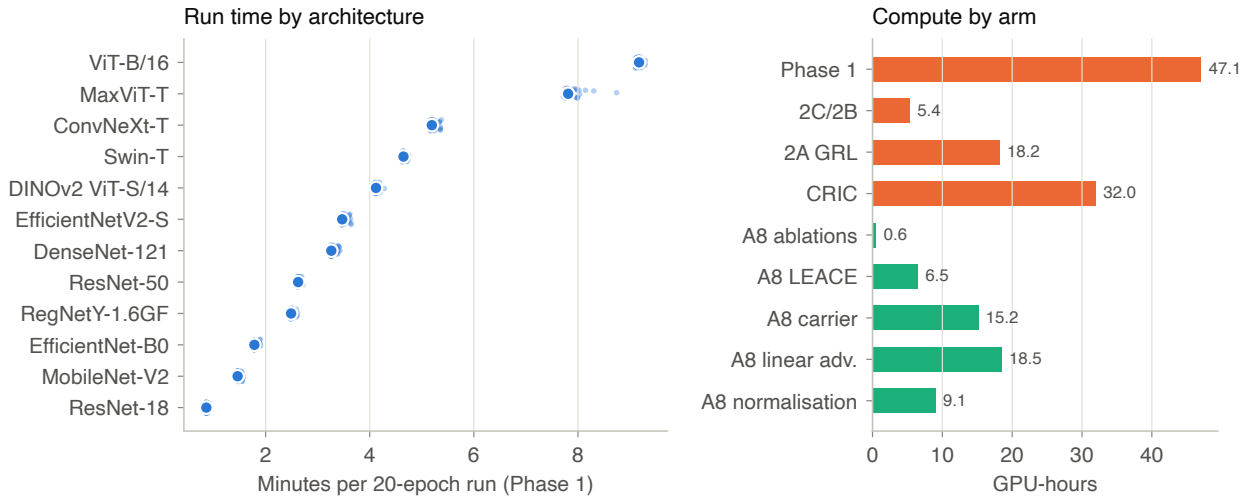


Figure 2.3: Left: wall time of each Phase 1 run by architecture (dots) and its median (ring). Right: GPU-hours by arm.

2.2.3 Software

Table 2.2: Software and hardware recorded in every training run.

Component	Version
python	3.13.9
torch	2.9.0+cu128
torchvision	0.24.0+cu128
timm	1.0.22
cuda	12.8
cudnn	91002
cuda_device	NVIDIA RTX A6000
numpy	2.5.2
pandas	3.0.5
scipy	1.18.1
scikit_learn	1.9.0
opencv	5.0.0
pillow	12.3.0
platform	Linux-4.18.0-372.9.1.el8.x86_64-x86_64-with-glibc2.28
cublas_workspace_config	:4096:8

2.3 Archiving and vacating

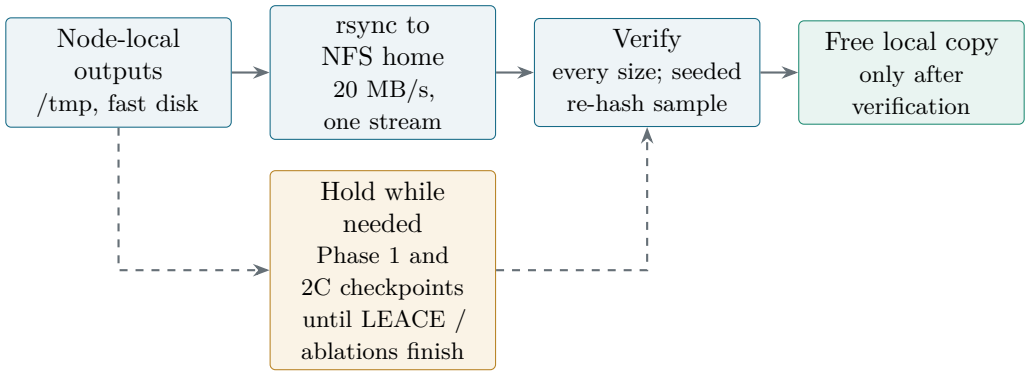


Figure 2.4: Offloading and archiving. The network home is slow and can stall, so copies are single, bandwidth-limited streams; nothing node-local is deleted until its archived copy has been verified.

The follow-up matrix would write about 52 GB of checkpoints into a 71 GB margin, so Study 2 checkpoints were moved to the network home while it ran: the gradient-reversal and CRIC checkpoints first, then the 2C checkpoints once all 160 ablation tasks had finished, then the Phase 1 checkpoints once all 480 LEACE tasks had finished. Each experiment was copied, every checkpoint's size compared, a seeded sample of fifty re-hashed against the hash in its run record, and only then deleted locally. At the end, the follow-up outputs were copied and verified (3,580 artifacts, sizes all matching, 100 re-hashed), the remaining Study 2 outputs re-verified (4,000 artifacts, 200 re-hashed), and the node-local working directory and shared-memory directory deleted. The archive (`~/cytofair_study2_archive`, 121 GB) holds every run record, prediction, feature file, checkpoint, log, and status file.

3

Errors, setbacks, and fixes

Study 1's setbacks (the invalid first classification matrix, the MAnet numerical failure, and the host bottleneck) are recorded in Chapter 15. Everything that went wrong afterwards is listed here, including mistakes in tooling and in procedure; none changed a registered model, split, seed, or endpoint.

Table 3.1: Every error and setback of Study 2 and Amendment 8, and its fix.

#	What went wrong	Fix	Effect on results
Study 2 preparation (before registration)			
1	No PDF text tools on the workstation	Python PDF libraries for reading and rendering	none
2	A recursive search over the slow network home stalled	stopped; rule: never scan the network home	none
3	`/scratch` not writable	node-local `/tmp` and `/dev/shm`	none
4	Four-core CPU quota starved data loaders	GPU-resident augmentation; heavy CPU analysis off-host	none (exact port, unit-tested)
5	Power script mis-named variance components; correlation on its -1 boundary	pattern matching; correlation set to 0 in the simulation	documented in the plan
6	R: S4 slot read with `$` in the protocol model	`@optinfo`	none
7	R: tidy-evaluation operator inside base `subset`	explicit indexing	none
8	2C budget of 240 crops per class infeasible for some seeds	220 crops, low/high ratio 0.70, feasible for 100/100 seeds	chosen before training
9	pandas 3 read-only arrays; Pillow deprecation	copies; argument removed	none
10	Non-deterministic attention backward (ViT)	math attention kernel only; bitwise repeat verified	none
11	Lint findings, a wrong test expectation	fixed	none
12	SSH launches hung on backgrounded command chains	`cd; setsid nohup ... & disown`	none
13	A shell wait loop did not split its input	explicit `read`	none
14	No rule for diverged adversarial runs	Amendment 7, before any 2A run	none (no divergence)
Study 2 analysis			

#	What went wrong	Fix	Effect on results
15	One float16 probe-feature row overflowed (EfficientNetV2-S, one crop)	documented; no follow-up arm reads it	one crop in a descriptive measure
16	A completion watcher never fired	replaced with a file-based check	none
Amendment 8			
17	Inference-mode tensors reused in autograd; non-deterministic `index_add_`	clone; one-hot matrix sums	caught in tests
18	R summary object not serialisable to JSON	converted to a list	caught in dry run
19	Remote `pgrep -f` matched its own command line	file-based completion checks	none
20	Interim preparer crawled on a GPU busy with training	stopped; registered run on a free GPU	none
21	**Preparer labelled standard carrier runs as colour-augmented** (`None` became a truthy NaN)	condition read from each run record; unit test; **Amendment 9**	script halted before output; no estimate seen
22	An archive fetch included smoke checkpoints	refetched records only	none
23	Dose figure plotted out-of-regime levels; labels overlapped	registered regimes only; layout fixed	presentation only
24	Negative numbers typeset with a hyphen	typographic minus in the macro generator	presentation only

The one that mattered

Error 21 is the only one that could have changed a reported number. The synthetic dry run had written condition labels directly and so never exercised the preparer's labelling. The R script stopped at F4 because the contrast term was missing, which exposed the bug before any output existed. The lesson recorded for the future: dry runs must go through every stage of the real pipeline, not start from the tables that stage produces.

Bibliography